In [1]:
```
#import libraries

import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder

from sklearn.metrics import mean_squared_error

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

In [2]:
```
#Load Data
dataset = pd.read_csv('House Price India.csv')
dataset = dataset.dropna()
```

In [3]:
```
#Data tranformation

le = LabelEncoder()

for col in dataset.columns:
    if dataset[col].dtype == 'object':
        dataset[col] = le.fit_transform(dataset[col])

dataset.head()
```

Out[3]:

	id	Date	number of bedrooms	number of bathrooms	living area	lot area	number of floors	waterfront present	number of views
0	6762810635	42491	4	2.50	2920	4000	1.5	0	0
1	6762810998	42491	5	2.75	2910	9480	1.5	0	0
2	6762812605	42491	4	2.50	3310	42998	2.0	0	0
3	6762812919	42491	3	2.00	2710	4500	1.5	0	0
4	6762813105	42491	3	2.50	2600	4750	1.0	0	0

5 rows × 23 columns

In [4]:
```
#Assign input & target columns

X = dataset.drop("Price", axis=1).values
y = dataset["Price"].values
```

In [5]:
```
#Split dataset

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta
```

In [6]:
```
#Feature scaling

sc = StandardScaler()

X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

In [7]:
```
# Build ANN

model = Sequential([
    Dense(32, activation='relu', input_shape=(X_train.shape[1],)),
    Dense(16, activation='relu'),
    Dense(1, activation='linear')
])
```

```
d:\Advanced Data Science Assignments\.venv\Lib\site-packages\keras\src\layers\core\d
ense.py:107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a lay
er. When using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

In [8]:
```
# Compile ANN

model.compile(
    optimizer='adam',
    loss='mse',
    metrics=['mae']
)
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for Ten
sorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use
WSL2 or the TensorFlow-DirectML plugin.
```

In [9]:
```
# Train ANN

history = model.fit(
    X_train,
    y_train,
    epochs=50,
    batch_size=16,
    validation_split=0.2
)
```

```
Epoch 1/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - loss: 423932952576.0000 - mae: 538947.937
5 - val_loss: 409884884992.0000 - val_mae: 535448.3750
Epoch 2/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 418916925440.0000 - mae: 535756.000
0 - val_loss: 400054419456.0000 - val_mae: 529123.3750
Epoch 3/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 402055069696.0000 - mae: 525272.125
0 - val_loss: 376308662272.0000 - val_mae: 513837.6250
Epoch 4/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 371751682048.0000 - mae: 505642.031
2 - val_loss: 340369408000.0000 - val_mae: 489622.7188
Epoch 5/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 329967009792.0000 - mae: 477164.531
2 - val_loss: 294255394816.0000 - val_mae: 456143.2812
Epoch 6/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 279600562176.0000 - mae: 439722.062
5 - val_loss: 241990467584.0000 - val_mae: 413843.5938
Epoch 7/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 225743454208.0000 - mae: 394179.531
2 - val_loss: 189647585280.0000 - val_mae: 364617.7812
Epoch 8/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 175258435584.0000 - mae: 343415.000
0 - val_loss: 144114450432.0000 - val_mae: 312717.9062
Epoch 9/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 133194727424.0000 - mae: 293052.593
8 - val_loss: 108167708672.0000 - val_mae: 263809.0000
Epoch 10/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 102242689024.0000 - mae: 250038.062
5 - val_loss: 84158914560.0000 - val_mae: 226775.9219
Epoch 11/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - loss: 82445213696.0000 - mae: 219992.9688
- val_loss: 69924667392.0000 - val_mae: 203069.7344
Epoch 12/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - loss: 71006748672.0000 - mae: 200685.4844
- val_loss: 61870055424.0000 - val_mae: 188254.1094
Epoch 13/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 64329728000.0000 - mae: 188518.3281
- val_loss: 57065291776.0000 - val_mae: 178546.1406
Epoch 14/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 60285808640.0000 - mae: 180435.0781
- val_loss: 53925085184.0000 - val_mae: 172148.9062
Epoch 15/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 57577902080.0000 - mae: 174974.7812
- val_loss: 51622240256.0000 - val_mae: 167606.9062
Epoch 16/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 55594835968.0000 - mae: 171136.8750
- val_loss: 49836273664.0000 - val_mae: 164233.9062
Epoch 17/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 53970444288.0000 - mae: 167756.6250
- val_loss: 48257490944.0000 - val_mae: 161274.7656
Epoch 18/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 52528820224.0000 - mae: 164675.5312
- val_loss: 46804783104.0000 - val_mae: 158526.8438
Epoch 19/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 51187544064.0000 - mae: 161885.3281
```

```
- val_loss: 45421400064.0000 - val_mae: 155802.2812
Epoch 20/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 49885007872.0000 - mae: 158751.5312
- val_loss: 44090789888.0000 - val_mae: 153138.8750
Epoch 21/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 48643751936.0000 - mae: 156294.2500
- val_loss: 42775007232.0000 - val_mae: 150377.0781
Epoch 22/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 47422779392.0000 - mae: 153419.0625
- val_loss: 41495392256.0000 - val_mae: 147609.9375
Epoch 23/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 46231740416.0000 - mae: 150305.3750
- val_loss: 40279257088.0000 - val_mae: 145043.3438
Epoch 24/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 45071384576.0000 - mae: 147688.4219
- val_loss: 39072264192.0000 - val_mae: 142282.2812
Epoch 25/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 43944767488.0000 - mae: 144728.7969
- val_loss: 37910208512.0000 - val_mae: 139694.2812
Epoch 26/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 42836443136.0000 - mae: 142044.5469
- val_loss: 36774907904.0000 - val_mae: 137070.4531
Epoch 27/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 41776476160.0000 - mae: 139004.8906
- val_loss: 35698745344.0000 - val_mae: 134573.1406
Epoch 28/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 40730243072.0000 - mae: 136431.9062
- val_loss: 34617581568.0000 - val_mae: 131796.1406
Epoch 29/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 39706288128.0000 - mae: 133324.9531
- val_loss: 33592369152.0000 - val_mae: 129290.3672
Epoch 30/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 38685044736.0000 - mae: 130420.5078
- val_loss: 32603402240.0000 - val_mae: 126765.7734
Epoch 31/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 37714546688.0000 - mae: 127985.7344
- val_loss: 31593943040.0000 - val_mae: 123836.5547
Epoch 32/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 36775428096.0000 - mae: 124606.0000
- val_loss: 30662356992.0000 - val_mae: 121267.7031
Epoch 33/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 35852025856.0000 - mae: 122182.3594
- val_loss: 29746835456.0000 - val_mae: 118466.0391
Epoch 34/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 34968391680.0000 - mae: 119453.5625
- val_loss: 28865374208.0000 - val_mae: 115519.0156
Epoch 35/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 34125406208.0000 - mae: 115919.4844
- val_loss: 28063705088.0000 - val_mae: 113047.2812
Epoch 36/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 33327558656.0000 - mae: 113570.0625
- val_loss: 27267782656.0000 - val_mae: 110279.2500
Epoch 37/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 32573747200.0000 - mae: 110916.0938
- val_loss: 26530924544.0000 - val_mae: 107581.4141
Epoch 38/50
```

```
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 31871291392.0000 - mae: 107904.5781
- val_loss: 25855371264.0000 - val_mae: 105131.5156
Epoch 39/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 31220512768.0000 - mae: 105627.7969
- val_loss: 25233522688.0000 - val_mae: 102716.1406
Epoch 40/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 30638129152.0000 - mae: 103046.2969
- val_loss: 24661520384.0000 - val_mae: 100442.8516
Epoch 41/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - loss: 30096631808.0000 - mae: 100893.3906
- val_loss: 24157065216.0000 - val_mae: 98545.9766
Epoch 42/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 4s 6ms/step - loss: 29615564800.0000 - mae: 98920.0234
- val_loss: 23691225088.0000 - val_mae: 96616.3672
Epoch 43/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 29185255424.0000 - mae: 96777.5625
- val_loss: 23285590016.0000 - val_mae: 95083.3125
Epoch 44/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 28818171904.0000 - mae: 95162.1484
- val_loss: 22922659840.0000 - val_mae: 93697.9688
Epoch 45/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 28469829632.0000 - mae: 94325.7109
- val_loss: 22566760448.0000 - val_mae: 91983.4219
Epoch 46/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 28158152704.0000 - mae: 92592.6562
- val_loss: 22268555264.0000 - val_mae: 90745.5391
Epoch 47/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 27888967680.0000 - mae: 91487.1406
- val_loss: 22003040256.0000 - val_mae: 89700.0625
Epoch 48/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 27627321344.0000 - mae: 90214.9609
- val_loss: 21764438016.0000 - val_mae: 88924.6016
Epoch 49/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 27402878976.0000 - mae: 89494.0234
- val_loss: 21543999488.0000 - val_mae: 88097.0234
Epoch 50/50
585/585 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 27181135872.0000 - mae: 89055.9297
- val_loss: 21310199808.0000 - val_mae: 86873.9453
```

In [10]:
```
#Predict

y_pred = model.predict(X_test)
```

```
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
```

In [11]:
```
#Evaluate

rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print("RMSE =", rmse)
```

```
RMSE = 172375.46071294486
```

In [12]:
```
results = model.evaluate(X_test, y_test)
print('Test Loss, Test MAE:', results)
```

```
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 29713287168.0000 - mae: 90392.6719
Test Loss, Test MAE: [29713287168.0, 90392.671875]
```

In [ ]:

Prajwal Mhase B.Tech IT 93

In [65]:
```
import numpy as np
import pandas as pd
```

In [66]:
```
df=pd.read_csv('data_1_B.csv')
```

In [67]:
```
df.head()
```

Out[67]:

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothn
0	842302	M	17.99	10.38	122.80	1001.0	
1	842517	M	20.57	17.77	132.90	1326.0	
2	84300903	M	19.69	21.25	130.00	1203.0	
3	84348301	M	11.42	20.38	77.58	386.1	
4	84358402	M	20.29	14.34	135.10	1297.0	

5 rows × 33 columns

In [68]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 33 columns):
 #   Column                   Non-Null Count  Dtype  
---  ------                   --------------  -----  
 0   id                       569 non-null    int64  
 1   diagnosis                569 non-null    str    
 2   radius_mean              569 non-null    float64
 3   texture_mean             569 non-null    float64
 4   perimeter_mean           569 non-null    float64
 5   area_mean                569 non-null    float64
 6   smoothness_mean          569 non-null    float64
 7   compactness_mean         569 non-null    float64
 8   concavity_mean           569 non-null    float64
 9   concave points_mean      569 non-null    float64
 10  symmetry_mean            569 non-null    float64
 11  fractal_dimension_mean   569 non-null    float64
 12  radius_se                569 non-null    float64
 13  texture_se               569 non-null    float64
 14  perimeter_se             569 non-null    float64
 15  area_se                  569 non-null    float64
 16  smoothness_se            569 non-null    float64
 17  compactness_se           569 non-null    float64
 18  concavity_se             569 non-null    float64
 19  concave points_se        569 non-null    float64
 20  symmetry_se              569 non-null    float64
 21  fractal_dimension_se     569 non-null    float64
 22  radius_worst             569 non-null    float64
 23  texture_worst            569 non-null    float64
 24  perimeter_worst          569 non-null    float64
 25  area_worst               569 non-null    float64
 26  smoothness_worst         569 non-null    float64
 27  compactness_worst        569 non-null    float64
 28  concavity_worst          569 non-null    float64
 29  concave points_worst     569 non-null    float64
 30  symmetry_worst           569 non-null    float64
 31  fractal_dimension_worst  569 non-null    float64
 32  Unnamed: 32              0 non-null      float64
dtypes: float64(31), int64(1), str(1)
memory usage: 146.8 KB
```

In [69]:
```
df.isnull().sum()
```

Out[69]:
```
id                         0
diagnosis                  0
radius_mean                0
texture_mean               0
perimeter_mean             0
area_mean                  0
smoothness_mean            0
compactness_mean           0
concavity_mean             0
concave points_mean        0
symmetry_mean              0
fractal_dimension_mean     0
radius_se                  0
texture_se                 0
perimeter_se               0
area_se                    0
smoothness_se              0
compactness_se             0
concavity_se               0
concave points_se          0
symmetry_se                0
fractal_dimension_se       0
radius_worst               0
texture_worst              0
perimeter_worst            0
area_worst                 0
smoothness_worst           0
compactness_worst          0
concavity_worst            0
concave points_worst       0
symmetry_worst             0
fractal_dimension_worst    0
Unnamed: 32              569
dtype: int64
```

In [70]:
```
df.drop(columns=['id', 'Unnamed: 32'], inplace=True)
```

In [71]:
```
df.describe()
```

Out[71]:

	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	comp
count	569.000000	569.000000	569.000000	569.000000	569.000000	
mean	14.127292	19.289649	91.969033	654.889104	0.096360	
std	3.524049	4.301036	24.298981	351.914129	0.014064	
min	6.981000	9.710000	43.790000	143.500000	0.052630	
25%	11.700000	16.170000	75.170000	420.300000	0.086370	
50%	13.370000	18.840000	86.240000	551.100000	0.095870	
75%	15.780000	21.800000	104.100000	782.700000	0.105300	
max	28.110000	39.280000	188.500000	2501.000000	0.163400	

8 rows × 30 columns

In [72]:
```
from sklearn.preprocessing import LabelEncoder
```

In [73]:
```
le = LabelEncoder()

df['diagnosis'] = le.fit_transform(df['diagnosis'])
```

In [74]:
```
df.head()
```

Out[74]:

	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean
0	1	17.99	10.38	122.80	1001.0	0.11840
1	1	20.57	17.77	132.90	1326.0	0.08474
2	1	19.69	21.25	130.00	1203.0	0.10960
3	1	11.42	20.38	77.58	386.1	0.14250
4	1	20.29	14.34	135.10	1297.0	0.10030

5 rows × 31 columns

In [75]:
```
X = df.drop('diagnosis', axis=1)
y = df['diagnosis']
```

In [76]:
```
from sklearn.model_selection import train_test_split
```

In [77]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y,test_size=0.2,random_state
```

In [78]:
```
from sklearn.preprocessing import StandardScaler
```

In [79]:
```
sc = StandardScaler()

X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

In [80]:
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

In [81]:
```
classifier = Sequential()
```

In [82]:
```
classifier.add(Dense(units=6, activation='relu', input_dim=X_train.shape[1]))
```

```
d:\Advanced data science Lab\.venv\Lib\site-packages\keras\src\layers\core\dense.py:
107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first layer in
the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

In [83]:
```
classifier.add(Dense(units=6, activation='relu'))
```

In [84]:
```
classifier.add(Dense(units=1, activation='sigmoid'))
```

In [85]:
```
classifier.compile(optimizer='adam',loss='binary_crossentropy',metrics=['accuracy']
```

In [86]:
```
classifier.fit(X_train,y_train,batch_size=23,epochs=56)
```

```
Epoch 1/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.5648 - loss: 0.7104
Epoch 2/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7165 - loss: 0.6419
Epoch 3/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8593 - loss: 0.5853
Epoch 4/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9187 - loss: 0.5287
Epoch 5/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9341 - loss: 0.4658
Epoch 6/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9341 - loss: 0.3962
Epoch 7/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9407 - loss: 0.3304
Epoch 8/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9429 - loss: 0.2744
Epoch 9/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9516 - loss: 0.2314
Epoch 10/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9582 - loss: 0.1997
Epoch 11/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9626 - loss: 0.1747
Epoch 12/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9626 - loss: 0.1563
Epoch 13/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9670 - loss: 0.1412
Epoch 14/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9670 - loss: 0.1293
Epoch 15/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9692 - loss: 0.1196
Epoch 16/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9714 - loss: 0.1118
Epoch 17/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9714 - loss: 0.1053
Epoch 18/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9758 - loss: 0.0995
Epoch 19/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9758 - loss: 0.0945
Epoch 20/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9780 - loss: 0.0902
Epoch 21/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9780 - loss: 0.0860
Epoch 22/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9802 - loss: 0.0825
Epoch 23/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9846 - loss: 0.0793
Epoch 24/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9846 - loss: 0.0766
Epoch 25/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9846 - loss: 0.0739
Epoch 26/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9868 - loss: 0.0714
Epoch 27/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9846 - loss: 0.0691
Epoch 28/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9846 - loss: 0.0669
```

```
Epoch 29/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9846 - loss: 0.0653
Epoch 30/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9846 - loss: 0.0633
Epoch 31/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9868 - loss: 0.0619
Epoch 32/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9868 - loss: 0.0603
Epoch 33/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9890 - loss: 0.0587
Epoch 34/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9890 - loss: 0.0574
Epoch 35/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9890 - loss: 0.0563
Epoch 36/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9890 - loss: 0.0548
Epoch 37/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9890 - loss: 0.0537
Epoch 38/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9890 - loss: 0.0525
Epoch 39/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9890 - loss: 0.0515
Epoch 40/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9890 - loss: 0.0505
Epoch 41/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9890 - loss: 0.0491
Epoch 42/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9890 - loss: 0.0484
Epoch 43/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9912 - loss: 0.0476
Epoch 44/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9912 - loss: 0.0469
Epoch 45/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9912 - loss: 0.0457
Epoch 46/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9912 - loss: 0.0449
Epoch 47/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9912 - loss: 0.0443
Epoch 48/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9934 - loss: 0.0435
Epoch 49/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9934 - loss: 0.0427
Epoch 50/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9934 - loss: 0.0419
Epoch 51/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9934 - loss: 0.0413
Epoch 52/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9934 - loss: 0.0408
Epoch 53/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9934 - loss: 0.0400
Epoch 54/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9934 - loss: 0.0391
Epoch 55/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9934 - loss: 0.0386
Epoch 56/56
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9934 - loss: 0.0381
```

Out[86]: <keras.src.callbacks.history.History at 0x1898c4927e0>

In [87]:
```
y_pred = classifier.predict(X_test)
```

```
WARNING:tensorflow:5 out of the last 9 calls to <function TensorFlowTrainer.make_pre
dict_function.<locals>.one_step_on_data_distributed at 0x000001898D69AD40> triggered
tf.function retracing. Tracing is expensive and the excessive number of tracings cou
ld be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors wit
h different shapes, (3) passing Python objects instead of tensors. For (1), please d
efine your @tf.function outside of the loop. For (2), @tf.function has reduce_retrac
ing=True option that can avoid unnecessary retracing. For (3), please refer to http
s://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorfl
ow.org/api_docs/python/tf/function for  more details.
1/4 ━━━━━━━━━━━━━━━━━━━━ 0s 49ms/stepWARNING:tensorflow:6 out of the last 12 calls t
o <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distri
buted at 0x000001898D69AD40> triggered tf.function retracing. Tracing is expensive a
nd the excessive number of tracings could be due to (1) creating @tf.function repeat
edly in a loop, (2) passing tensors with different shapes, (3) passing Python object
s instead of tensors. For (1), please define your @tf.function outside of the loop.
For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary re
tracing. For (3), please refer to https://www.tensorflow.org/guide/function#controll
ing_retracing and https://www.tensorflow.org/api_docs/python/tf/function for  more d
etails.
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step
```

In [88]:
```
y_pred = (y_pred > 0.5)
```

In [89]:
```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)

print(cm)
```

```
[[70  1]
 [ 2 41]]
```

In [90]:
```
from sklearn.metrics import accuracy_score
```

In [91]:
```
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
```

```
Accuracy: 0.9736842105263158
```